

DSC 140B

Representation Learning

Lecture 16 | Part 1

The Final Project

Final Project

- ▶ Released yesterday on `dsc140b.com`
- ▶ **Goal:** train a classifier to predict SoCal city where street view images were taken
- ▶ **Two parts:**
 - ▶ **Model:** autograded for accuracy
 - ▶ **Short report:** graded for effort and completeness

Final Project

- ▶ Starter code is available (link in writeup).
 - ▶ Shows how to export a PyTorch model.
- ▶ Due Wednesday of Finals week (Wed., March 18).
 - ▶ Can use slip day.
- ▶ Start early!

DSC 140B

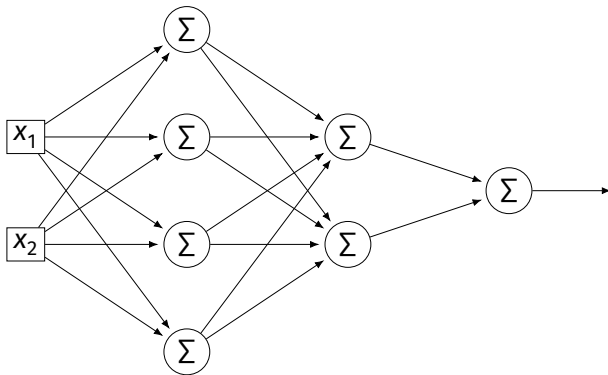
Representation Learning

Lecture 16 | Part 2

Multiple Outputs

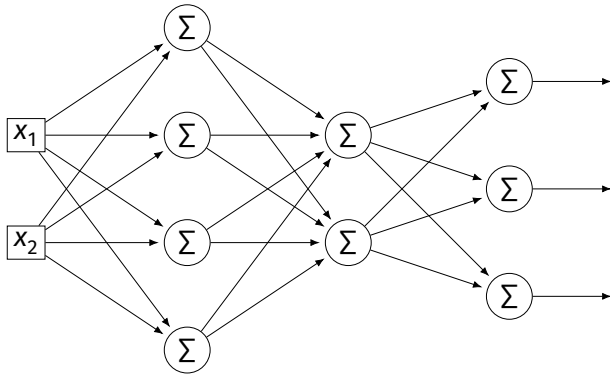
So far...

- We have seen networks with a single output.



Multiple Outputs

- NNs can have **multiple** output nodes, too.



Why?

- ▶ It is sometimes useful to predict multiple things at once.
- ▶ **Regression:**
 - ▶ Given weather history, predict temp. and humidity.
 - ▶ Given image, predict latitude and longitude.
- ▶ **Classification:**
 - ▶ Given image, predict if it contains a cat, dog, horse, penguin, squirrel, etc.

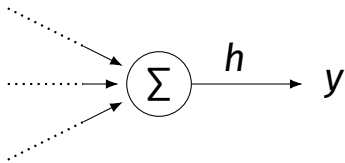
Why?

- ▶ *Could* train separate NNs, one for each output.
 - ▶ One network for temperature, one for humidity.
- ▶ However, training one network takes advantage of **shared structure**.

How?

- ▶ Most aspects of training stay the same.
- ▶ However, we need to think more carefully about the **loss function**.
 - ▶ **Before:** prediction was a number, $h = H(\vec{x})$.
 - ▶ **Now:** prediction is a vector, $\vec{h} = H(\vec{x})$.

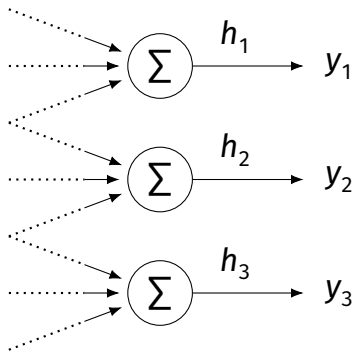
Single-target Regression



- With one output, we often use square loss:

$$\ell(h, y) = (h - y)^2$$

Multi-target Regression



- ▶ With K outputs, use the sum of square losses:

$$\ell(\vec{h}, \vec{y}) = \sum_{k=1}^K (h_k - y_k)^2$$

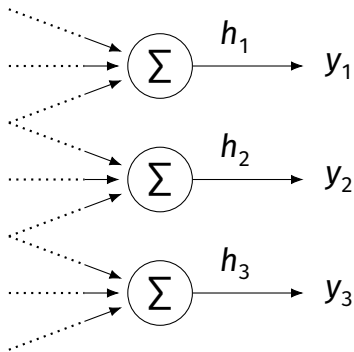
- ▶ Equivalently:

$$\ell(\vec{h}, \vec{y}) = \|\vec{h} - \vec{y}\|^2$$

Classification

- ▶ There are two situations.
- ▶ **Mutually Exclusive:** only one output is “on”.
 - ▶ E.g., image contains either a cat, dog, horse, penguin, *or* squirrel (but not multiple animals at once).
 - ▶ A.k.a., **multi-class classification**.
- ▶ **Non-Mutually Exclusive:** any subset can be “on”.
 - ▶ E.g., image can contain a cat, dog, horse, penguin, *and* squirrel at the same time.
 - ▶ A.k.a., **multi-label classification**.

Non-Mutually Exclusive

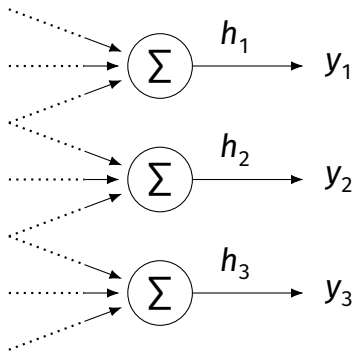


- Activation: **sigmoid**.
- Interpretation: h_k is the predicted probability that $y_k = 1$.

Exercise

True or False: the predicted probabilities h_1 , h_2 , and h_3 must sum to 1 when using sigmoid activations.

Non-Mutually Exclusive



► Loss: **binary cross-entropy**.

$$\ell(\vec{h}, \vec{y}) = - \sum_{k=1}^K \begin{cases} \log h_k, & \text{if } y_k = 1 \\ \log [1 - h_k], & \text{if } y_k = 0 \end{cases}$$

Exercise

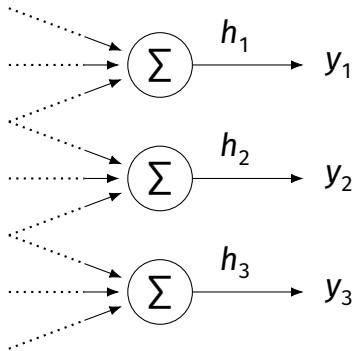
Suppose the true label is $\vec{y} = (1, 0, 1)$ and the predicted probabilities (sigmoid outputs) are $\vec{h} = (0.9, 0.2, 0.8)$.

What is the binary cross-entropy loss?

In PyTorch

- ▶ In PyTorch, use `nn.BCEWithLogitsLoss`.
- ▶ Combines sigmoid activation and binary cross-entropy loss; more numerically stable.
- ▶ **Do not** apply sigmoid activation to the output layer if using `nn.BCEWithLogitsLoss`.
 - ▶ Use linear activation.
 - ▶ The output is said to be measured in **logits**.

Mutually Exclusive



- ▶ We want only one output to be “on” at a time.
- ▶ Can encourage this with the **softmax** activation:

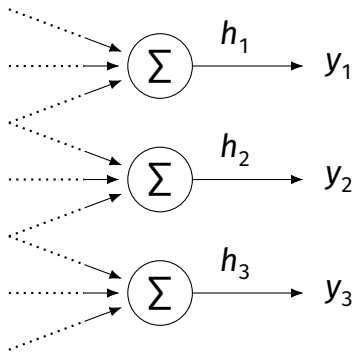
$$h_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

- ▶ Interpretation: h_k is the predicted probability that $y_k = 1$ and $y_j = 0$ for all $j \neq k$.

Exercise

True or False: the predicted probabilities h_1 , h_2 , and h_3 must sum to 1 when using softmax activations.

Mutually Exclusive



► Loss function: **categorical cross-entropy**.

$$\ell(\vec{h}, \vec{y}) = - \sum_{k=1}^K \begin{cases} \log h_k, & \text{if } y_k = 1 \\ 0, & \text{if } y_k = 0 \end{cases}$$

Exercise

Suppose the true label is $\vec{y} = (0, 1, 0)$ and the predicted probabilities (softmax outputs) are $\vec{h} = (0.2, 0.5, 0.3)$.

What is the categorical cross-entropy loss?

In PyTorch

- ▶ In PyTorch, use `nn.CrossEntropyLoss`.
- ▶ Combines softmax activation and categorical cross-entropy loss; more numerically stable.
- ▶ **Do not** apply softmax activation to the output layer if using `nn.CrossEntropyLoss`.

DSC 140B

Representation Learning

Lecture 16 | Part 3

Autoencoders

Generalizing PCA

- ▶ We started the quarter with PCA.
- ▶ PCA is a **linear** method.
- ▶ We can generalize upon PCA to derive nonlinear representation learners.

Representation Learning

- ▶ At a high level, representation learning finds an **encoding function** $\text{encode}(\vec{x}) : \mathbb{R}^d \rightarrow \mathbb{R}^k$.
- ▶ Ideally, this function captures useful aspects of the data distribution.

Example: PCA

- ▶ In PCA, we **encode** a point $\vec{x}$ by projecting it onto the top k eigenvectors of data covariance matrix:

$$\text{encode}(\vec{x}) = U^T \vec{x}$$

Decoding

- ▶ Encoding can decrease dimensionality.
- ▶ Intuitively, we may want to preserve as much “information” about $\vec{x}$ as possible.
- ▶ We should be able to **decode** the encoding and **reconstruct** the original point, approximately.

$$\vec{x} \approx \text{decode}(\text{encode}(\vec{x}))$$

Example: PCA

- ▶ In PCA, given a point $\vec{z} \in \mathbb{R}^k$ in the new representation, the **reconstruction** is:

$$\text{decode}(\vec{z}) = U\vec{z}$$

Representation Learning

- ▶ **Goal:** find an encoder (and decoder) such that

$$\text{encode}(\text{decode}(\vec{x})) \approx \vec{x}$$

Reconstruction Error

- ▶ In general, $\text{decode}(\text{encode}(\vec{x}))$ will not be exactly equal to $\vec{x}$.
- ▶ One way of quantifying the difference w.r.t. data is the (ℓ_2) **reconstruction error**:

$$\sum_{i=1}^n \|\vec{x}^{(i)} - \text{decode}(\text{encode}(\vec{x}^{(i)}))\|^2$$

Note

- ▶ Of course, it is trivial to find an encoder/decoder with zero reconstruction error:

$$\text{encode}(\vec{x}) = \vec{x} = \text{decode}(\vec{x})$$

- ▶ Such an encoder is not useful.
- ▶ Instead, we constrain the form of the encoder so that it cannot simply copy the input.

Example: PCA

- ▶ Assume $\text{encode}(\vec{x}) = U\vec{x}$, for some matrix U whose $k \leq d$ columns are orthonormal.
 - ▶ That is, the encoding is an orthogonal projection.
- ▶ **Goal:** find U to minimize reconstruction error on a dataset $\vec{x}^{(1)}, \dots, \vec{x}^{(d)}$.
- ▶ **Solution:** pick columns of U to be top k eigenvectors of data covariance matrix.

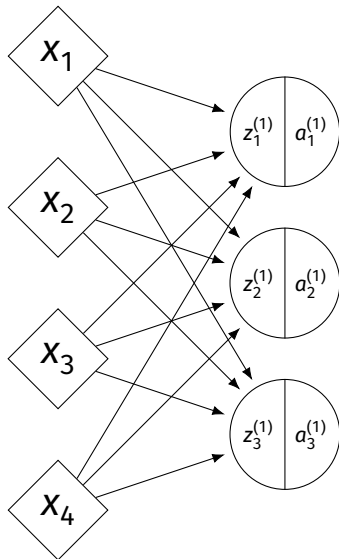
Now

- ▶ $\text{encode}(\vec{x}) = U\vec{x}$ is a linear encoding function.
- ▶ What if we let encode be nonlinear?
- ▶ That is, let's generalize PCA.

Encoder as a Neural Network

- ▶ Assume $\text{encode}(\vec{x})$ is a (deep) **neural network**.
- ▶ Output is not a single number, but k numbers.
 - ▶ I.e., a vector in $\mathbb{R}^k$
- ▶ Can use nonlinear activations, have more than one layer.

Encoder as a Neural Network



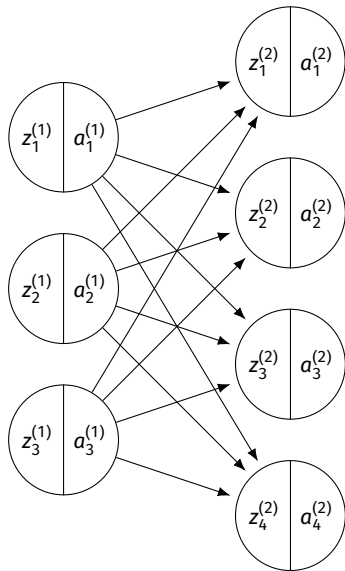
Encoder as a Neural Network

- ▶ The output of the encoder is the new representation.
- ▶ To train the encoder, we'll need a **decoder**.

Decoder as a Neural Network

- ▶ Assume $\text{decode}(\vec{z})$ is a (deep) **neural network**.
- ▶ Output is not a single number, but d numbers.
 - ▶ Same dimensionality as original input, $\vec{x}$.
 - ▶ I.e., a vector in $\mathbb{R}^d$
- ▶ Can use nonlinear activations, have more than one layer.

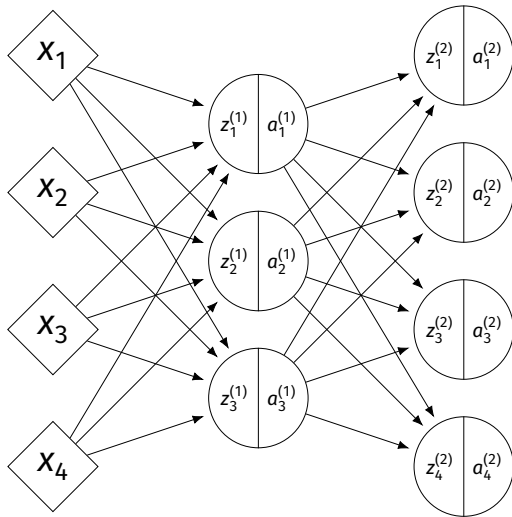
Decoder as a Neural Network



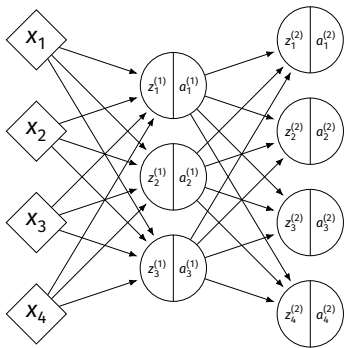
decode(encode($\vec{x}$)) as a **NN**

- ▶ Together, decode(encode($\vec{x}$)) is a neural network $H(\vec{x}) : \mathbb{R}^d \rightarrow \mathbb{R}^d$.

decode(encode($\vec{x}$)) as a NN



Training



- We want $H(\vec{x}) \approx \vec{x}$
- One approach: train network to minimize reconstruction error.

$$\begin{aligned}\sum_{i=1}^n \|\vec{x}^{(i)} - H(\vec{x}^{(i)})\|^2 &= \sum_{i=1}^n \sum_{j=1}^d (\vec{x}_j^{(i)} - (H(\vec{x}^{(i)}))_j)^2 \\ &= \sum_{i=1}^n \sum_{j=1}^d (\vec{x}_j^{(i)} - a_j^{(2)}(\vec{x}^{(i)}))^2\end{aligned}$$

Training

- ▶ The network can be trained using gradient-based methods.
 - ▶ E.g., stochastic gradient descent.
- ▶ **Note:** this is an **unsupervised** learning problem.

Autoencoders

- ▶ When the encoder/decoder are NNs,
 $H(\vec{x}) = \text{decode}(\text{encode}(\vec{x}))$ is an **autoencoder**.

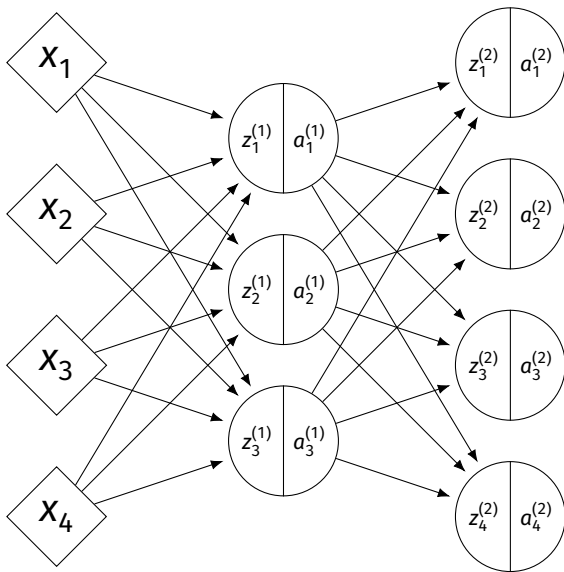
Generalizing PCA

- ▶ We can view autoencoders as generalizations of PCA.
- ▶ Consider again the encoder that performs an orthogonal projection:

$$\text{encode}(\vec{x}) = U^T \vec{x}$$

$$\text{decode}(\vec{z}) = U \vec{z}$$

- ▶ encode/decode are neural networks (with linear activations).



Exercise

True/False: training an autoencoder to minimize reconstruction error will result in the same $\text{encode}(\vec{x})$ function as PCA.

Answer: False

- ▶ PCA minimizes reconstruction error **subject to** the constraint that the columns of U are orthonormal.
- ▶ Without the orthonormality constraint, the autoencoder learns a different encoding.
- ▶ *However*, the autoencoder learns a (non-orthogonal) projection into the same space as PCA.

In other words...

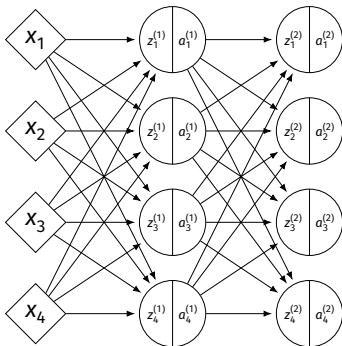
- ▶ PCA is an autoencoder trained with an additional orthonormality constraint.
- ▶ Cannot easily be learned by gradient descent; find eigenvectors instead.

Uses of Autoencoders

- ▶ Like PCA, autoencoders can be used for **dimensionality reduction**.
- ▶ Unlike PCA, autoencoders can learn nonlinear maps.
- ▶ Encoded data can be used as input to predictive model, etc.

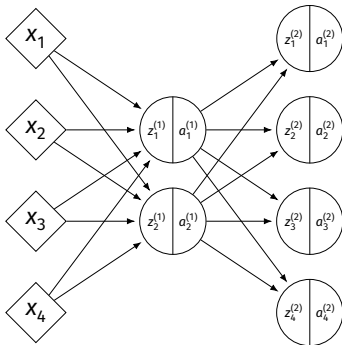
Dimensionality Reduction

- If the dimensionality of the encoder is the same as the dimensionality of $\vec{x}$, the autoencoder can learn to simply reproduce the input.



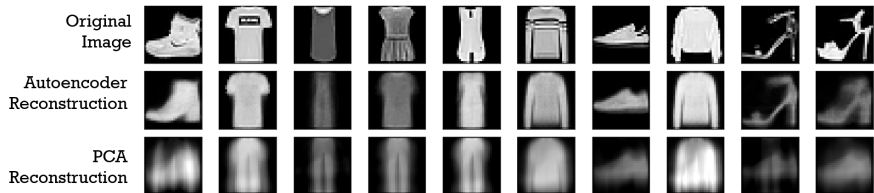
Dimensionality Reduction

- As such, we choose number of hidden nodes $< d$.



- Called an **undercomplete autoencoder**.

Example

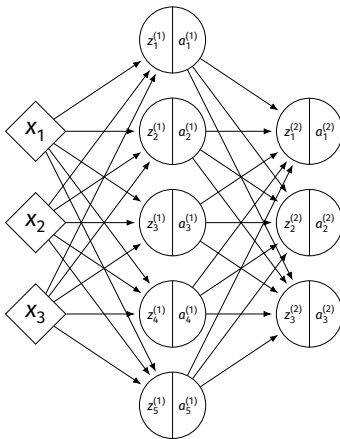


1

¹By Michela Massi - Own work, CC BY-SA 4.0,
<https://commons.wikimedia.org/w/index.php?curid=80176900>

Other Uses

- However, sometimes it is useful for hidden layer to have **greater** dimensionality.



Denoising Autoencoders

- ▶ One such case is in **denoising autoencoders**.
- ▶ **Idea:** train an autoencoder to remove noise.
- ▶ Add random noise to each $\vec{x}^{(i)}$ to get $\tilde{x}^{(i)}$.
- ▶ Train network so that $H(\tilde{x}^{(i)}) \approx \vec{x}$.